

BASES DE DATOS

UNIDAD 2: DISEÑO DE BASES DE DATOS MODELO RELACIONAL

AUTOR

Esp. Lic. Fabiana María Riva

DISEÑO DE BASES DE DATOS – MODELO RELACIONAL

Diseño Curricular de la Carrera de Ingeniería en Sistemas de Información

Plan 2023 - Ordenanza 1877

Contenidos Mínimos s/Ordenanza

Diseño de Bases de Datos

Álgebra Relacional

OBJETIVOS s/Ordenanza

Emplear metodologías de diseño de Bases de Datos Conceptuales

COMPETENCIAS RELACIONADAS s/Ordenanza

Competencia específica de la carrera:

CE 1.1. Especificar, proyectar y desarrollar sistemas de información.

Competencias genéricas de la carrera:

CG 1. Identificar, formular y resolver problemas de ingeniería.

CG 4. Utilizar de manera efectiva las técnicas y herramientas de aplicación en la ingeniería.

CG 8. Fundamentos para una actuación ética y responsable.

CG9 Fundamentos para evaluar y actuar en relación con el impacto social de su actividad profesional en el contexto global y local.

CG 10. Aprender en forma continua y autónoma.

RESULTADO DE APRENDIZAJE Y CAPACIDADES RELACIONADAS DEFINIDAS:

RA1: Construir el modelo conceptual de datos a partir de una especificación de un proceso organizacional con la finalidad de abstraer las estructuras de datos necesarias para almacenar y obtener datos de un sistema de información.

3. Capacidad para diseñar el Modelo Relacional a partir del Modelo Entidad-Relación o utilizando el proceso de Normalización: Identificación de tablas, atributos, claves primarias y foráneas.
4. Capacidad para resolver las consignas referidas a la obtención de datos del modelo utilizando Álgebra Relacional.

INDICE

Modelo Relacional	1
Dependencias Funcionales y Claves	2
Normalización	8
Transformación de un Modelo E-R en Relacional	13
Reglas de transformación	13
Análisis comparativo de los enfoques de conversión de jerarquías de generalización ...	20
Algebra Relacional	24
Definición y clasificación	24
Operaciones Fundamentales	25
Operaciones Adicionales o Derivadas.....	32
Inserción – Eliminación - Actualización	38
Anexo I – Ejemplo Modelo Relacional con datos.....	40
Anexo II – Ejemplo de Combinación Complejo	42

MODELO RELACIONAL

El modelo relacional es un modelo basado en la lógica de predicados y en la teoría de conjuntos.

Sus fundamentos fueron planteados por Edgar Codd y se constituyó rápidamente en el modelo por excelencia que se trasladó a la implementación de Bases de Datos, en este caso **Bases de Datos Relacionales**.

Como hemos mencionado al introducir el concepto de Modelos de Datos, estos son una combinación de tres componentes: Estructuras de Datos, Reglas de Integridad, Operadores.

En el Modelo Relacional, estos tres componentes son:

Estructura de Datos: Tablas o Relaciones. Formalmente, el nombre de Modelo Relacional deriva del uso de relaciones (o tablas) que constan de un conjunto de t-uplas (filas de la tabla o registros) constituidas por un conjunto de atributos (columnas de la tabla) cuyos valores pertenecen a un dominio.

A la cantidad de atributos que constituyen una relación se lo denomina grado de la relación.

Reglas de Integridad o Restricciones: Para el Modelo Relacional las reglas de integridad pueden diferenciarse en:

- **Integridad de Entidad:** establece que se requiere de la definición de una **clave primaria** o identificador.

Una **clave primaria** es un atributo o conjunto mínimo de atributos de una relación o tabla que identifica unívocamente al resto de los atributos de la relación.

En función de esto la clave primaria debe tener un valor único para el conjunto de filas de la tabla, ninguno de sus atributos puede admitir valor nulo y no deben existir atributos que no agreguen información a la clave (atributos superfluos).

Una clave primaria se selecciona del conjunto de **claves candidatas** que puede tener una relación o tabla.

- **Integridad Referencial:** Esta restricción surge de definir una **clave foránea** en una tabla.

Una **clave foránea** es un atributo o conjunto de atributos de una relación o tabla que está relacionado con la clave primaria de otra relación o tabla. Una clave foránea puede admitir valores nulos (todos sus atributos) pero si tiene valor, éste debe ser coincidente con la clave primaria de la tabla con la que está relacionada. Con esto se

asegura la integridad entre la clave foránea de una tabla (a la que llamaremos hija) y la clave primaria de la tabla (que llamaremos padre) con la que se relaciona.

- **Integridad de Dominio:** Dominio de valores de los atributos que constituyen la relación o tabla, es decir los valores que pueden admitirse en el atributo, aquí trabajaremos con el concepto de dominio semántico del atributo (es decir su significado). A partir del Capítulo 3, donde trabajaremos con la implementación en bases de datos, será el tipo de datos con el que definiremos cada atributo o columna de la tabla.

DEPENDENCIAS FUNCIONALES Y CLAVES

Las **dependencias funcionales** están ligadas a la búsqueda de **claves** y es necesaria su comprensión para desarrollar el proceso de **normalización** y definir el modelo relacional que se implementará luego en una base de datos relacional.

Definiciones

ESQUEMA DE RELACIÓN: $r = R (T, DF)$

donde:

R: nombre de la relación

T: conjunto de atributos que componen la relación: $T = \{A1, A2, A3, ..., An\}$

n: grado de la relación

DF: conjunto de restricciones o **dependencias funcionales** que se deducen de las reglas del dominio que estamos trabajando:

DF = {DF1; DF2;; DFm}

Cada **DFi** se expresa como $X \rightarrow Y$

donde:

X e **Y** son subconjuntos del conjunto de atributos T de R y se denominan **descriptores**

X se denomina **determinante** e **Y** se denomina **determinado**.

Se dice que **Y es funcionalmente dependiente de X** o que **X determina funcionalmente a Y**.

Entonces:

Sea **R** una **relación**, **X** e **Y** subconjuntos arbitrarios del conjunto de atributos de **R** a los que denominaremos **descriptores**. Se dice que **Y es funcionalmente dependiente de X** (**$X \rightarrow Y$**) si todo valor legal posible de **X** tiene asociado precisamente un único valor de **Y**

Supongamos el siguiente ejemplo:

El conjunto de atributos (**T**) que componen la relación será:

EXAMEN (NroLegajo, ApellidoyNombres, CodAsig, NombreAsig, LegajoDoc, NombreDoc, FechaExamen, Nota)

Y la siguiente dependencia funcional:

Df1:

NroLegajo, CodAsig, FechaExamen $\rightarrow$ ApellidoyNombres, NombreAsig, LegajoDoc, NombreDoc, NroActa, Nota

Que expresa que: dado un Legajo de un alumno, el código de la asignatura y la fecha de examen se pueden identificar el Nombre del Alumno, el Nombre de la Asignatura, el Legajo del Docente presidente de la Mesa examinadora y su nombre y la Nota que obtuvo el alumno.

Además, se tiene que:

Df2: NroLegajo → ApellidoyNombres

Df3: CodAsig → NombreAsig

Df4: LegajoDoc → NombreDoc

Df5: NroActa → CodAsig, FechaExamen

Df6: CodAsig, FechaExamen → NroActa

Diremos que Df1 es la **clausura** de las dependencias funcionales (**DF⁺**) ya que a partir de este conjunto de atributos (que además es mínimo) se pueden obtener todos los atributos de la relación que estamos analizando (veremos más adelante los axiomas que hacen posible esto).

Además, diremos que:

En la dependencia funcional Df1 hay **dependencias parciales** ya que existen atributos que no dependen totalmente del determinante: Df2 y Df3

Hay una **dependencia transitiva**: Df4 en la dependencia funcional Df1

Podríamos agregar el atributo DNI del alumno al esquema de relación del ejemplo obteniendo:

EXÁMENES (NroLegajo, CodAsig, FechaExamen, ApellidoyNombres, DNI, NombreAsig, LegajoDoc, NombreDoc, NroActa, Nota)

Pudiendo agregar la siguiente dependencia funcional al esquema:

Df1': DNI, CodAsig, FechaExamen → NroLegajo, ApellidoyNombres, NombreAsig, LegajoDoc, NombreDoc, NroActa, Nota

Es decir que ahora en el ejemplo hay dos conjuntos de atributos que son la **clausura** de las dependencias funcionales **DF⁺**: Df1 y Df1'

Y además se pueden agregar las siguientes dependencias funcionales:

Df2': NroLegajo → ApellidoyNombres, DNI

Df2'': DNI → ApellidoyNombres, NroLegajo

Aquí aplica el concepto de claves dado anteriormente:

Una **clave candidata**: es el atributo o conjunto mínimo de atributos que identifica unívocamente una instancia de una relación o tabla (o, lo que es lo mismo, el conjunto mínimo de atributos tales que su clausura es igual a todos los atributos de la relación).

En función de esto **ninguno** de los atributos que conforman una clave candidata podrá admitir valores nulos, tampoco los atributos podrán ser superfluos, es decir no deben existir atributos que no agreguen información a la clave (concepto de minimalidad).

De todas las claves candidatas se seleccionará una para utilizarla como clave de la relación y la denominaremos **clave primaria**.

En el ejemplo anterior hemos partido de dependencias funcionales que conformaban claves candidatas ($Df_1 - Df_1'$ y $Df_2' - Df_2''$), sin embargo, es posible que las mismas no se encuentren definidas a simple vista y haya que obtenerlas.

El trabajo para la obtención de claves tiene una base matemática compleja que puede ser utilizada en dominios complejos y puede ser consultada en la bibliografía especificada por la Cátedra.

De la misma utilizaremos los Axiomas de Armstrong y nos concentraremos en trabajar sobre ejemplos para facilitar su comprensión.

Axiomas de Armstrong

Supongamos el siguiente ejemplo:

El conjunto de atributos (T) que componen la relación será:

ARTICULOS (CodArticulo, DescArticulo, CodTalle, DescTalle, CodColor, DescColor, CodBarra),

Y las dependencias funcionales (DF):

$CodArticulo \rightarrow DescArticulo, CodTalle, CodColor, CodBarra$

$CodBarra \rightarrow DescArticulo, CodTalle, CodColor, CodArticulo$

$DescArticulo \rightarrow CodArticulo, CodTalle, CodColor, CodBarra$

$CodTalle \rightarrow DescTalle$

$CodColor \rightarrow DescColor$

Reflexividad: $\forall Z \supseteq Y, Z \rightarrow Y$

Es decir, Si Y es un subconjunto de Z, entonces $Z \rightarrow Y$ con lo cual: $\forall Y, Y \rightarrow Y$

A partir de cualquier atributo o conjunto de atributos siempre puede deducirse él mismo.
(Dependencia trivial)

$CodArticulo \rightarrow CodArticulo \quad CodBarra \rightarrow CodBarra$

Aumentatividad: $\{X \rightarrow Y, Z \supseteq X\} \Rightarrow Z \rightarrow Y$

Se puede aumentar trivialmente el **determinante** de una dependencia.

$CodArticulo \rightarrow DescArticulo$, entonces $CodArticulo, CodTalle \rightarrow DescArticulo$

Transitividad o enlace de dependencias funcionales. $X \rightarrow Y$ e $Y \rightarrow Z \Rightarrow X \rightarrow Z$.

CodArticulo $\rightarrow$ CodTalle, DescTalle
CodTalle $\rightarrow$ DescTalle
entonces por transitividad CodArticulo $\rightarrow$ DescTalle

Éste es el mecanismo básico de funcionamiento del enlace entre tablas o relaciones o **claves foráneas**, como veremos más adelante

De estos se derivan:

Proyectividad o Descomposición: $\{ X \rightarrow Y, Z \subseteq Y \} \Rightarrow X \rightarrow Z$

CodArticulo $\rightarrow$ DescArticulo, CodTalle, CodColor, CodBarra
Se puede descomponer:
CodArticulo $\rightarrow$ DescArticulo
CodArticulo $\rightarrow$ CodTalle
CodArticulo $\rightarrow$ CodColor
CodArticulo $\rightarrow$ CodBarra

Aditividad o Unión: $\{ X \rightarrow Y, Z \rightarrow V \} \Rightarrow X \cup Z \rightarrow Y \cup V$

CodArticulo $\rightarrow$ DescArticulo
CodTalle $\rightarrow$ DescTalle
CodArticulo, CodTalle $\rightarrow$ DescArticulo, DescTalle

Pseudo-Transitividad: $\{ X \rightarrow Y, W \cup Y \rightarrow Z \} \Rightarrow X \cup W \rightarrow Z$.

CodArticulo $\rightarrow$ DescArticulo
DescArticulo, CodColor $\rightarrow$ DescColor
CodArticulo, CodColor $\rightarrow$ DescColor

Estos axiomas además pueden ser utilizados para:

1. Determinar si una dependencia funcional $X \rightarrow Y$ no indicada inicialmente en un conjunto DF es válida.
2. Determinar si dos representaciones del mismo problema son equivalentes
- 3. Determinar las claves candidatas y primarias**

En el ejemplo dado y a partir de aplicar los Axiomas de Armstrong recursivamente (algoritmo de Ullman) se puede llegar a las siguientes conclusiones:

Tomando:

CodArticulo $\rightarrow$ DescArticulo, CodTalle, CodColor, CodBarra

Por reflexividad: CodArticulo $\rightarrow$ CodArticulo

Por aditividad: CodArticulo $\rightarrow$ CodArticulo, DescArticulo, CodTalle, CodColor, CodBarra

Además: CodTalle $\rightarrow$ DescTalle

Por transitividad: CodArticulo $\rightarrow$ DescTalle

que se puede agregar a la dependencia funcional anterior quedando:

CodArticulo $\rightarrow$ CodArticulo, DescArticulo, CodTalle, DescTalle, CodColor, CodBarra

Lo mismo podemos hacer con la dependencia CodColor → DescColor quedando:

CodArticulo → CodArticulo, DescArticulo, CodTalle, DescTalle, CodColor, DescColor, CodBarra

Entonces, CodArticulo determina **TODOS** los atributos de la relación ARTICULOS del ejemplo por lo que se dice que CodArticulo es la clausura de las dependencias funcionales dadas y conforma una **clave candidata** de ARTICULOS

El mismo ejercicio puede hacerse con CodBarra y con DescArticulo que, en este ejemplo, serán también **claves candidatas**.

De estas claves candidatas se selecciona una que será utilizada como **clave primaria**.

No necesariamente se llega a que, dado un dominio y una vez identificados sus atributos y dependencias funcionales, se encuentra un único atributo o conjunto de atributos del cual dependen funcionalmente el resto de los atributos del dominio como en el ejemplo dado. Tanto en estos casos como para evitar otros problemas, una relación puede necesitar ser descompuesta en varias relaciones a través del proceso de normalización.

NORMALIZACIÓN

Normalización es el proceso por el cual una relación se “descompone” en dos o más relaciones que satisfacen ciertas condiciones denominadas **formas normales**.

Se normaliza para:

- evitar la redundancia de datos,
- evitar anomalías de inserción, eliminación y actualización de los datos en las tablas,
- proteger la integridad de datos,
- mejorar la independencia de los datos de los programas de aplicación (ampliaremos este concepto en el capítulo de Bases de Datos Relacionales).

En el proceso de normalización no se pierde información y gracias a esto se puede volver hacia atrás y recuperar la forma normal desde la que se partió.

Partiendo de la relación EXÁMENES vista anteriormente (recordar el concepto de clave candidata y clave primaria analizado en el ejemplo):

EXÁMENES (LegajoAlu, CodAsig, FechaExamen, DNIALu, NombreAlu, NombreAsig, CP, LegajoDoc, NombreDoc, NroActa, Nota)

NroLegajo	CodAsig	Fecha Examen	DNI	ApellidoyNombres	NombreAsig	LegajoDoc	NombreDoc	NroActa	Nota
36589	ISI-21	6/2/2018	37449777	J.Perez	G.Datos	11888	G.Suarez	10	7
34564	IQ-20	9/2/2018	40541118	L.Gimenez	Q.Orgánica	28333	S.Rodriguez	12	8
35777	ISI-20	8/2/2018	38459767	N.Ramirez	M.Superior	26777	H.Garay	11	6
41222	ISI-21	6/2/2018	39567822	A.Bianco	G.Datos	11888	G.Suarez	10	9
36589	IQ-20	8/2/2018	37449777	J.Perez	M.Superior	26777	H.Garay	11	9
.....									

En esta relación existe **redundancia de datos** lo que da lugar a las siguientes **anomalías**:

- **Anomalía de inserción:** Para insertar una nueva tupla en la relación EXÁMENES se deberán consignar los datos completos del alumno, la asignatura y el docente presidente de la mesa examinadora generando más redundancia de datos y la posibilidad de incurrir en un error de integridad de datos si no se repitan las dependencias funcionales establecidas

- **Anomalía de eliminación:** Si se eliminara en el ejemplo la instancia de examen para el alumno de legajo 35777, no tendríamos cómo recuperar los datos ni del alumno ni de la asignatura
- **Anomalía de actualización:** Si tuviéramos que cambiar el presidente de mesa de Gestión de Datos para el 6/2/2018, habría que recorrer toda la relación modificando cada instancia de exámenes donde esta mesa examinadora aparezca

Realizaremos entonces el proceso de normalización para evitar estos inconvenientes.

Primera Forma normal (1NF)

Se dice que una relación (tabla) está en la **Primera Forma Normal (1NF)** si cumple la propiedad que **"sus atributos o columnas contienen solo valores atómicos (no son multivaluados o compuestos) y pertenecen a un mismo dominio de valores"**

En el caso del ejemplo, podríamos considerar que los atributos CodAsig y FechaExamen no son atributos atómicos: CodAsig puede descomponerse en CodCarrera y NroAsig y FechaExamen en DiaExamen, MesExamen, AñoExamen.

Considerar un atributo como atómico depende de la implementación y de las aplicaciones en que utilizaremos el modelo.

Por ejemplo, si fuera necesario considerar cuáles son las carreras para las que se tomó examen sería necesario descomponer el atributo CodAsig, o si necesitáramos especificar los exámenes por mes sería necesario descomponer el atributo FechaExamen.

Suponiendo necesario descomponer el atributo CodAsig, la relación EXÁMENES **no está en la 1NF**, debe descomponerse el atributo CodAsig.

EXÁMENES (LegajoAlu, CodCarrera, NroAsig, FechaExamen,
CP

NombreAlu, NombreAsig, LegajoDoc, NombreDoc, NroActa, Nota)

Ahora la relación está en la **1NF**.

Segunda Forma normal (2NF)

Una relación (tabla) está en **Segunda Forma Normal (2NF)** si: ***"además de estar en 1NF cualquiera de sus atributos no-claves dependen completamente de cada una de las claves candidatas de la relación (tabla)"***

La dependencia debe ser completa sobre la totalidad de los atributos que conforman la clave (para claves compuestas) y no para una parte de ella. 2NF se aplica sólo cuando tenemos claves compuestas.

En el ejemplo, existían dependencias parciales identificadas con estas dependencias funcionales:

Df2': LegajoAlu → NombreAlu, DNIALu

Df3: CodCarrera, NroAsig → NombreAsig

Se requiere descomponer la relación para obtener la 2NF. Al descomponer estas relaciones se requiere además aplicar el concepto de integridad referencial, es decir, la definición de las claves foráneas.

Marcaremos las claves foráneas de la siguiente forma:

Atrib1, Atrib2, ..., AtribN → CF(NOMBRE_RELACION(Atribcf1, Atribcf2,..., AtribcfN))

Si Atrib1, Atrib2,...AtribN tienen los mismos nombres que Atribcf1, Atribcf2, ..., AtribcfN no hará falta mencionar estos últimos

ALUMNOS (LegajoAlu, NombreAlu, DNIALu)
CP

ASIGNATURAS (CodCarrera, NroAsig, NombreAsig)
CP

EXÁMENES (LegajoAlu, CodCarrera, NroAsig, FechaExamen, LegajoDoc, NombreDoc, NroActa, Nota)
CP

LegajoAlu → CF(ALUMNOS)

CodCarrera, NroAsig → CF(ASIGNATURAS)

Tercera Forma normal (3NF)

Una relación (tabla) está en **Tercera Forma Normal (3NF)** si: ***"además de estar en 2NF cualquiera de sus atributos no-claves no depende transitivamente de las claves candidatas de la relación (tabla)"***

En el ejemplo existe una dependencia transitiva expresada por:

Df6: LegajoDoc → NombreDoc

En función de esto se requiere una nueva descomposición de la relación MESAS_EXAMENES quedando el modelo en la 3NF de la siguiente forma:

ALUMNOS (LegajoAlu, NombreAlu, DNIALu)

CP

ASIGNATURAS (CodCarrera, NroAsig, NombreAsig)

CP

DOCENTES (LegajoDoc, NombreDoc)

CP

EXÁMENES (LegajoAlu, CodCarrera, NroAsig, FechaExamen, LegajoDoc, NroActa, Nota)

CP

LegajoAlu → CF(ALUMNOS(LegajoAlu))

CodCarrera, NroAsig → CF(ASIGNATURAS)

LegajoDoc → CF(DOCENTES) NN

Para las claves foráneas cuyos atributos no pertenecen a la clave primaria deben indicarse si admite o no valores nulos (NULL/NN)

Forma normal BOYCE-CODD (BCNF)

Una relación (tabla) está en la **BCNF** si: **"todo determinante es una clave candidata"**

Es una definición más rígida que la 3NF y se aplica en los casos que:

- 1) Una parte de una clave determina otra parte de una clave
- 2) Un atributo que no conforma parte de la clave determina parte de la clave

En el caso del ejemplo tenemos que en EXÁMENES:

NroActa → CodCarrera, NroAsig, FechaExamen por lo tendríamos:

ALUMNOS (LegajoAlu, NombreAlu, DNIALu)

CP

ASIGNATURAS (CodCarrera, NroAsig, NombreAsig)

CP

DOCENTES (LegajoDoc, NombreDoc)

CP

ACTAS (NroActa, CodCarrera, NroAsig, FechaExamen, LegajoDoc)

CP

LegajoDoc → CF(DOCENTES) NN

CodCarrera, NroAsig → CF(ASIGNATURAS) NN

EXÁMENES (LegajoAlu, NroActa, Nota)

LegajoAlu → CF(ALUMNOS)

NroActa → CF(ACTAS)

Es muy común que, dado un esquema relacional con sus dependencias funcionales no necesitemos realizar el proceso de normalización completo. Generalmente y sobre todo a partir de la práctica e identificando bien las dependencias funcionales, obtendremos un modelo normalizado, en la tercera forma normal (3NF) y solo nos restarán agregar algunas relaciones para las dependencias funcionales que no hayan quedado expresadas en el modelo (BCNF).

Existen otras formas normales que no derivan de las dependencias funcionales (4NF, 5NF) y que pueden ser analizadas de la bibliografía recomendada por la cátedra.

TRANSFORMACIÓN DE UN MODELO E-R EN RELACIONAL

El modelo relacional también puede ser obtenido a partir de la transformación de un modelo E-R en tablas relacionales.

Es importante la obtención de un Modelo Relacional para luego ser implementado en un SGDB (Sistema de Gestión de Bases de Datos) relacional.

Veremos que a partir de este método obtendremos un modelo en la 3NF, en ocasiones también la BCNF. Para ejemplificar el método de transformación utilizaremos el ejemplo expuesto en la Figura 12.

REGLAS DE TRANSFORMACIÓN

1. Regla del Tipo de Entidad: cada tipo de entidad se convierte en una tabla:

Las **entidades fuertes** del modelo se transforman en una tabla que incluye todos los atributos de la entidad. La clave primaria estará formada por el/los atributos que identifiquen unívocamente a cada instancia de la entidad.

Las **entidades débiles** del modelo se transforman en una tabla que incluye todos los atributos de la entidad más la clave del vínculo identificador (el/los atributos de la entidad fuerte de la que depende). La clave primaria de la tabla estará formada por la clave de la entidad de la que depende (relacionada con su vínculo identificador) más el atributo discriminante de la entidad débil. La clave de la entidad de la que depende debe marcarse como clave foránea.

NOTA: Los atributos que conforman clave primaria se subrayan y se indica CP.

Los atributos que conforman la clave foránea se especifican:

atributo1, atributo2, ..., atributoN → CF (NOMBRE_TABLA)

además se indica si la misma acepta nulos (NULL) o no los acepta (NOT NULL)

En el caso de que los atributos de la clave foránea pertenezcan a una clave primaria no es necesario indicar NOT NULL

Algunas entidades fuertes son: CATEGORIAS, ZONAS Y ARTICULOS

CATEGORIAS (CodCategoria, DescCategoria)
CP

ZONAS (CodZona, DescripciónZona)
CP

ARTICULOS (CodArticulo, DescripciónArticulo, Presentación)
CP

Como entidad débil encontramos a: ARTICULOS

PRECIOS (CodArticulo, FechaDesde, Precio)
CP

CodArticulo → CF(ARTICULOS)

- 2. Regla del vínculo binario 1 a M:** cada vínculo 1 a M se convierte en una clave foránea en la tabla que corresponda al tipo de entidad hija (lado con cardinalidad M). Si la cardinalidad mínima del lado de la entidad madre es 1 la clave foránea no puede aceptar valores nulos.

CLIENTES (CUIT, RazónSocial, Calle, Número, Piso, Departamento, CodCategoria, CodZona)

CP

CodCategoria → CF(CATEGORIAS) NULL

CodZona → CF(ZONAS) NN

- 3. Regla del vínculo binario M a N:** Cada vínculo binario M a N se convierte en una tabla separada conformada por los atributos que conforman la clave a cada lado del vínculo más los atributos descriptivos del vínculo.

Los atributos que conforman la clave a cada lado del vínculo conformarán las claves foráneas a cada una de las tablas derivadas de las entidades a cada lado del vínculo.

Para seleccionar la clave primaria:

Si el vínculo no posee atributos descriptivos: La clave primaria de la tabla es una combinación formada por las claves primarias de los tipos de entidad que participan en la relación.

Si el vínculo posee atributos descriptivos: la elección de clave primaria dependerá de las dependencias funcionales que surjan de la narrativa ya que puede ocurrir que alguno de los atributos de la relación deban o no formar parte de la clave.

En el ejemplo de la Figura 12, *VENDEDORES_ZONAS* es un vínculo binario N a M con atributos.

Recordemos la regla para deducir las dependencias funcionales y encontrar las claves candidatas:

- ✓ *Cada zona tiene asignado un único vendedor y un vendedor atiende una única zona desde una fecha y por un período determinado, es decir hasta una fecha. Puede ser que un vendedor vuelva a atender una misma zona en diferentes períodos.*

CodVendedor, FechaDesde → CodZona, FechaHasta

CodZona, FechaDesde → CodVendedor, FechaHasta

En este caso los dos determinantes encontrados pueden ser claves candidatas, no hay posibilidad de quitar ningún atributo. Seleccionaremos arbitrariamente la primera. Después veremos que es posible identificar ciertos requerimientos que permitirán una mejor selección de la clave primaria.

La tabla queda entonces:

VENEDORES_ZONAS (CodVendedor, FechaDesde, CodZona, FechaHasta)
CP

CodVendedor → CF(VENEDORES) NN

(La entidad VENEDORES se pasa a tabla en la regla del vínculo recursivo.)

CodZona → CF(ZONAS) NN

En el ejemplo de la Figura 18, existe un vínculo binario N a M entre *INSTALACIONES* y *SERVICIOS*. Las entidades vinculadas se convierten en tablas según la primer regla y el vínculo *INSTALACIONES_SERVICIOS* también se convierte en tabla. En este caso no es necesario analizar las dependencias funcionales para determinar la clave primaria.

INSTALACIONES (CodInstalación, NombreInstalación)
CP

SERVICIOS (CodServicio, DescripcionServicio)
CP

INSTALACIONES_ SERVICIOS(CodInstalación, CodServicio)
CP

CodInstalación → CF(INSTALACIONES)

CodServicio → CF(SERVICIOS)

Para completar el ejemplo, deberíamos pasar a tabla la entidad débil: VALORES que depende de la agregación *INSTALACIONES_SERVICIOS*: Esta entidad débil se pasa igual que lo indica la Regla 1.

VALORES (CodInstalación, CodServicio, Año, Mes, Monto)
CP

CodInstalación, CodServicio → CF(INSTALACIONES_SERVICIOS)

CUIDADO!!! La clave foránea es a la agregación.

4. **Regla del vínculo Recursivo:** Pueden aplicarse las reglas para vínculos 1 a N o N a M como se ve en los siguientes ejemplos:

En el ejemplo de la Figura 12 existen 2 vínculos recursivos: COMPONENTES y RESPONSABLE:

COMPONENTES se convierte con la regla del vínculo binario N a M en una nueva tabla con los atributos que conforman la clave de las entidades a cada lado del vínculo. En este caso sería CodArtículo, CodArtículo. Como no pueden repetirse nombres de atributos en una tabla se modifica uno quedando:

COMPONENTES (CodArtículo, CodArticuloComponente)
CP

CodArtículo → CF(ARTICULOS)

CodArticuloComponente → CF(ARTICULOS(CodArtículo))

Cuando los nombres de los atributos a los que hace referencia la clave foránea no son iguales se los menciona.

RESPONSABLE se convierte con la regla del vínculo 1 a N en una CF en la tabla VENDEDORES.

VENDEDORES (**NroLegajo**, ApellidoNombres, Direccion, Telefono,
NroLegajoResponsable)
CP

NroLegajoResponsable → CF(VENDEDORES(NroLegajo)) NULL

5. Vínculos de grado mayor a 2:

La regla coincide con la de vínculos N a M con atributos y deben analizarse según la narrativa.

Los vínculos ternarios se transforman en una tabla, cuyos campos son los atributos que componen las claves primarias de las entidades que relaciona y, si los tuviera, los atributos propios del vínculo.

Cada una de las claves primarias de las entidades que relaciona el vínculo ternario son a su vez claves foráneas.

La clave primaria del vínculo ternario debe definirse a través de un análisis de dependencias funcionales a partir de la narrativa.

En el ejemplo de la Figura 12 existe el vínculo ternario SOLICITUDES.

Recordemos las reglas:

- ✓ *Los clientes realizan solicitudes de artículos en una fecha y por una cantidad.*
- ✓ *Cada solicitud de artículos, en el momento que se realiza, es registrada por un vendedor.*

Hemos pasado a tablas CLIENTES, ARTICULOS y VENDEDORES.

Sabemos además que debemos incluir en la tabla SOLICITUDES: CUIT, FechaSolicitud, CodArtículo y NroLegajo.

Tratemos de deducir las dependencias funcionales.

Decir **CUIT → FechaSolicitud, CodArticulo, cantidad, NroLegajo** significaría que un cliente puede realizar una única solicitud en una fecha de un artículo por una cantidad y ésta estará registrada por un vendedor (recordar que los determinados son únicos para cada valor del determinante).

Esto no cumple con las reglas indicadas y de la misma forma se puede deducir que ningún atributo individual puede ser clave de esta tabla.

Decir **CUIT, FechaSolicitud → CodArticulo, cantidad, NroLegajo** significaría que un cliente puede realizar muchas solicitudes en distintas fechas, pero en esa fecha solo podrá solicitar un único artículo por una cantidad y ésta estará registrada por un vendedor, pero las reglas dicen que puede realizar solicitudes de artículos (muchos).

Decir **CUIT, FechaSolicitud, CodArticulo → cantidad, NroLegajo** significaría que un cliente puede realizar muchas solicitudes en distintas fechas de muchos artículos, y de cada solicitud solicitará una cantidad y estará registrada por un vendedor.

Esto coincide con las reglas indicadas y CUIT, FechaSolicitud, CodArtículo conforman la clave primaria de la tabla.

Un error muy común es intentar incluir en la clave todos los atributos clave de las entidades vinculadas más los atributos del vínculo. De hacer esto para el ejemplo nos quedaría la siguiente dependencia:

CUIT, FechaSolicitud, CodArticulo, NroLegajo → cantidad

Esto significaría que la solicitud de un artículo en una fecha puede ser registrada por muchos vendedores que no coincide con las reglas indicadas.

En función de lo dicho la tabla quedará:

SOLICITAN (CUIT, CodArticulo, Fecha_Solicitud, NroLegajo, Cantidad)
 CP

CUIT → CF(CLIENTES)

CodArticulo → CF(ARTICULOS)

NroLegajo → CF(VENDEDORES) NN

6. Conversión de Jerarquías de generalización: La conversión de Jerarquías de generalización pueden permitir varias alternativas, todas válidas. En cada caso particular se deberán analizar las ventajas y desventajas de cada una y seleccionar aquella más conveniente al caso particular. Se pueden tomar los siguientes enfoques:

a. Conversión de jerarquía completa:

En este enfoque cada entidad de la jerarquía se transforma en una tabla siguiendo la Regla 1, con los atributos indicados en cada superclase/subclase. Además se indican en las subclases la clave primaria de la superclase que se convierte en clave foránea a la tabla de la jerarquía de la que dependen.

TODAS las tablas creadas tienen LA MISMA CLAVE PRIMARIA, la de la SUPERCLASE.

Aplicando este enfoque al ejemplo de la Figura 14 la conversión de la jerarquía de artículos resultaría en:

ARTICULOS (IdArticulo, Descripcion, TipoArticulo)
 CP

BEBIDAS (IdArticulo, CantCm3)
 CP

IdArticulo → CF(ARTICULOS)

INGREDIENTES (IdArticulo, UnidadMedida)
 CP

IdArticulo → CF(ARTICULOS)

ELABORADOS (IdArticulo, Receta)
 CP

IdArticulo → CF(INGREDIENTES)

COMPRADOS (IdArticulo, CantDiasVto)
 CF(INGREDIENTES)
 CP

IdArticulo →

Aquí cada tabla contiene los atributos propios y todas las tablas, la clave de la superclase.

Analizar las claves foráneas.

b. Conversión a Superclase

En este caso la conversión resulta en una única tabla que surge de la transformación de la superclase a la que se le agregan los atributos de todas las subclases, pudiendo ser los mismos nulos. En este caso la clave primaria de la tabla es la clave primaria de la superclase.

Normalmente puede determinarse si una entidad es de un tipo de subclase dentro de la jerarquía verificando que un atributo propio de la subclase, que sea obligatorio, no sea nulo para esa entidad. Sin embargo, excepcionalmente, puede que esto no sea posible debido a que la subclase no tiene un atributo propio obligatorio. En ese caso, para identificar la subclase a la que pertenece la entidad, puede agregarse un campo llamado *Tipo* a la tabla que será útil cuando la jerarquía que estamos transformando es disyunta.

Aplicando este enfoque a la jerarquía de artículos del ejemplo de la Figura 14 obtenemos la siguiente conversión:

ARTICULOS(IdArticulo, Descripcion, TipoArticulo, CantCm3, UnidadMedida, Receta, CantDiasVto)

CP

En este caso todos los atributos de las subclases admiten valores nulos aunque en el dominio del modelo E-R no sea así.

Para saber si es un ingrediente elaborado debemos revisar si la cantidad de días de vencimiento es no nula.

Para conocer si es un ingrediente comprado debemos revisar si la receta es no nula o ver si ese artículo puede ser comprado a un proveedor.

Para determinar si es un ingrediente simplemente controlamos que la unidad_medida no sea nula. Para saber si es una bebida chequeamos que la Cant_cm3 no sea nula.

c. Conversión a Subclases

Este enfoque consiste en que sólo las subclases de menor nivel se implementan.

Las mismas heredan todos los atributos de sus superclases que no se implementan.

Al igual que en el enfoque 6.a. Conversión de jerarquía completa donde TODAS las tablas comparten LA MISMA CLAVE PRIMARIA, que corresponde a la CP de la superclase.

Continuando con el ejemplo de la Figura 14 la conversión resultante sería:

BEBIDAS (IdArticulo, Descripción, CantCm3)

CP

INGREDIENTE_ELABORADO (IdArticulo, Descripción, UnidadMedida, Receta)

CP

INGREDIENTE_COMPRADO (IdArticulo, Descripción, UnidadMedida, cantDiasVto)

CP

ANÁLISIS COMPARATIVO DE LOS ENFOQUES DE CONVERSIÓN DE JERARQUÍAS DE GENERALIZACIÓN

Anteriormente se presentaron tres enfoques de conversión de jerarquías de generalización. Aunque todos ellos sean válidos para todos los casos no siempre serán apropiados o convenientes. Analizaremos a continuación las ventajas y desventajas.

Conversión de Jerarquía completa

Ventajas

- Mejor uso de espacio de almacenamiento.
- Modelización exacta del dominio.

Desventajas

- Para acceder a toda la información de una subclase debe accederse a las tablas de todas sus superclases. Para nuestro ejemplo para obtener toda la información de un ingrediente elaborado habría que acceder a las tablas de ARTICULOS, INGREDIENTES y ELABORADOS.
- Requiere coordinación entre las claves primarias de las superclases y subclases.
- Cada nuevo subtipo requiere al menos una nueva tabla y las consultas asociadas a la misma.
- Complejo modelado de las jerarquías solapadas.
- Complejo mecanismo de identificación del tipo de la subclase

Conversión a superclase

Ventajas

- El acceso a cualquier subclase sólo requiere acceso a una tabla.
- No requiere coordinación entre las claves primarias de las superclases y subclases.
- Fácil modelado de jerarquías solapadas.
- Rara vez requiere modificaciones en el modelo de datos para agregar una nueva subclase.

Desventajas

- Desperdicio de espacio para aquellos atributos de las subclases que no son comunes.
- Complejo mecanismo de identificación de tipo de la subclase.

Conversión a subclases

Ventajas

- Fácil mecanismo de identificación de la subclase
- Uso de espacio de almacenamiento eficiente cuando los atributos comunes de las subclases son pocos.
- Eficiente acceso a todos los datos de una subclase particular.

Desventajas

- Uso de espacio de almacenamiento ineficiente cuando los atributos comunes de las subclases son muchos.
- Requiere mecanismos complejos de coordinación de claves primarias para jerarquías solapadas.
- Cada nuevo subtipo requiere una nueva tabla y las consultas asociadas a la misma.
- Complejo modelado de las jerarquías solapadas
- Mecanismo de identificación de tipo de la subclase más complejo.

En cada modelo en particular se deberán considerar las ventajas y desventajas de cada enfoque y evaluar en cada caso cual resulta más conveniente ya que este tipo de conversión suele tener un impacto sumamente importante en el costo de mantenimiento de los sistemas.

EXCEPCIÓN A LOS TRES ENFOQUES: CATEGORÍAS Y RETÍCULAS

El modelo relacional provee un soporte limitado al proceso de Generalización-Especialización

A través de los tres enfoques antes presentados pueden modelarse la mayoría de los casos, sin embargo hay casos particulares donde deben aplicarse otros mecanismos de transformación, en función de las restricciones que plantea el modelo relacional.

Los problemas que surgen de la implementación de este tipo de Generalización-Especialización son descritos en bibliografía sobre ORM (Object Relational Mapping), es nombrado como Implementación de Herencia Múltiple y se presenta principalmente en la Categorías y Retículas

CATEGORÍAS

El principal problema surge de que las entidades que se agrupan son de naturaleza distinta y por tal motivo es común que las claves primarias de las entidades sean incompatibles.

Por ejemplo de la Figura 16 *"El bar atiende clientes particulares o empresas que pueden tener cuenta corriente"*. Los conjuntos de entidades EMPRESAS y PARTICULARES cuentan con distinta clave primaria, es decir, difieren en el dominio (incluso al implementarlo en base de datos podrían diferir los tipos de datos de las mismas).

En estos casos la alternativa es transformar las superclases en tablas y agregarles las columnas de la subclase a cada superclase. Esto sería equivalente al enfoque de **Conversión a Superclase**.

Sin embargo, en los casos donde la clave primaria no sea incompatible podrían usarse los enfoques Conversión de jerarquía completa o a subclase antes explicados.

Supongamos que en el ejemplo anterior se utilizara un número único y secuencial (NroCliente) para la clave primaria de las EMPRESAS y de los PARTICULARES. En este caso debe cuidarse que los números de EMPRESAS y PARTICULARES no coincidan. Imagine qué pasaría si se usa el número 1 como clave de una empresa y como clave de un particular. Por ello debe implementarse un mecanismo para que estos números nunca puedan coincidir.

Alternativamente puede agregarse un nuevo atributo de (TipoCliente) como parte de la clave para evitar la superposición entonces la Clave primaria de la entidad Clientes en cuenta corriente **sería**

TipoCliente, NroCliente
CP

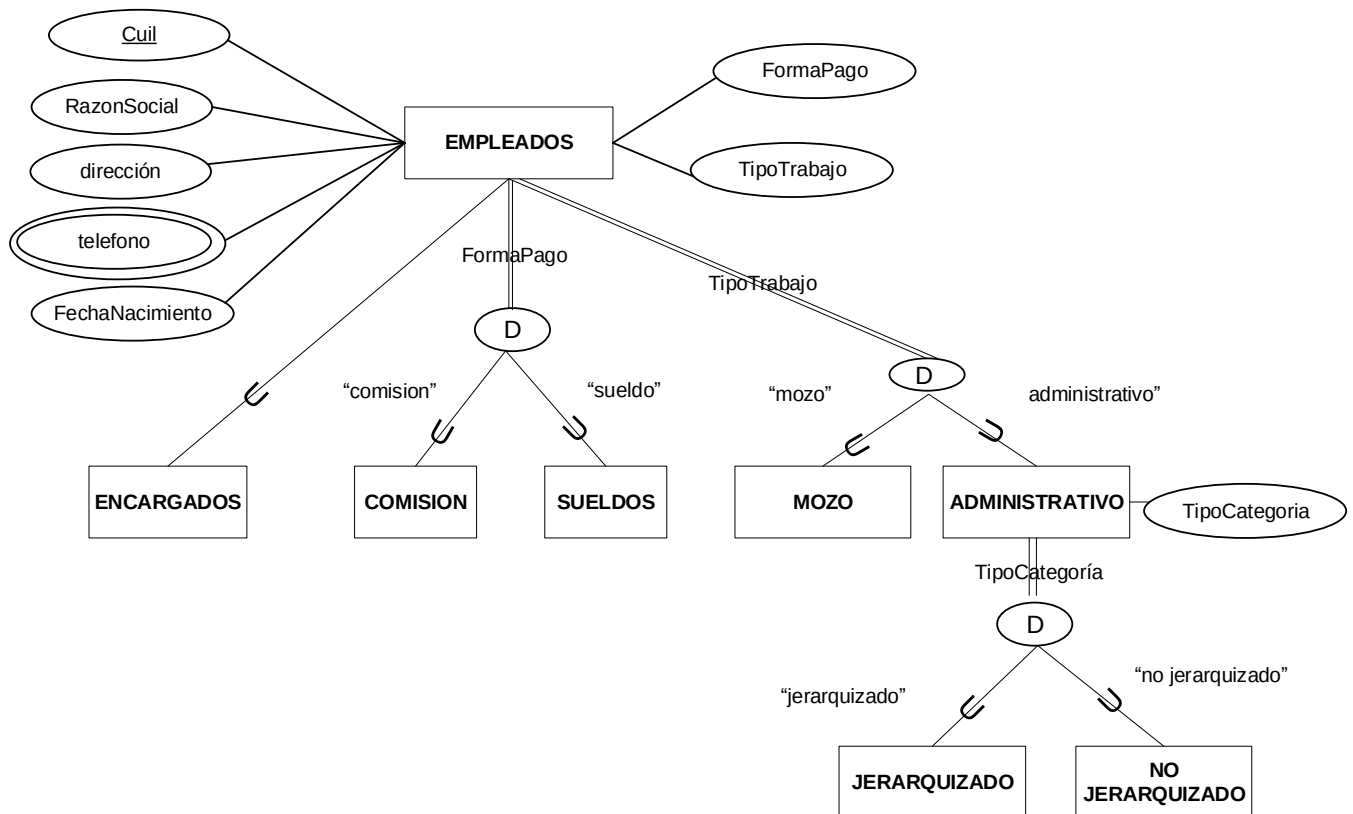
RETÍCULAS

La conversión de una retícula puede dividirse en dos etapas: Etapa de Transformación de Jerarquía y Etapa de Transformación de Herencia múltiple.

Etapas de Transformación de Jerarquía

Se considera desde la superclase de mayor jerarquía hasta todas las subclases que tengan herencia simple y se aplican los tres enfoques antes explicados

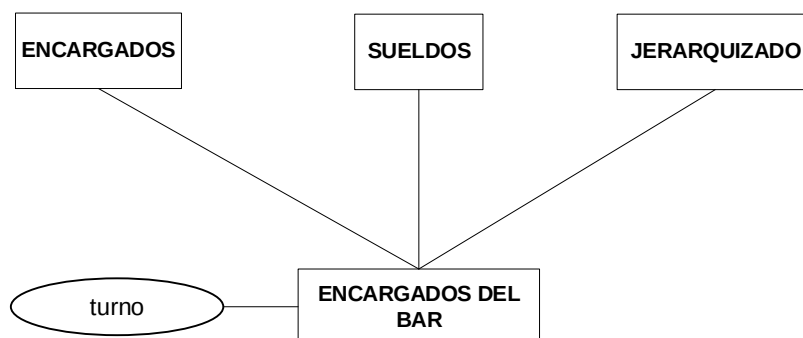
Para el ejemplo de la Figura 13 sería:



Donde el tipo de entidades ENCARGADOS DEL BAR se ha eliminado del análisis. Luego se pasa a la siguiente etapa

Etapas de Transformación de Herencia múltiple

En esta etapa se analizan las subclases con herencia múltiple y las superclases de las que hereda:



Entonces se aplica el mismo análisis que se utilizó para las Categorías

ALGEBRA RELACIONAL

DEFINICIÓN Y CLASIFICACIÓN

El álgebra relacional es un conjunto de operaciones definidas para manipular datos en el modelo relacional. Permitirán especificar las peticiones de recuperación o actualización de datos. Es un lenguaje procedimental.

Las operaciones se aplicarán sobre una o más relaciones del modelo y darán como resultado una nueva relación. Estas nuevas relaciones podrán formar parte de nuevas operaciones.

Existen varias clasificaciones de las operaciones mencionadas.

Una clasificación es la que diferencia los operadores que aplican nociones de la teoría de conjunto y las que no. La teoría matemática de conjuntos es posible de aplicar ya que las relaciones se definen como un conjunto de tuplas. Entre estas operaciones que aplican la teoría de conjunto se incluyen: UNION, INTERSECCION, DIFERENCIA y PRODUCTO CARTESIANO. El resto de operaciones, definidas específicamente para manipular el modelo, se encuentran: SELECCIÓN, PROYECCIÓN Y REUNION (o JOIN).

Otra clasificación se realiza en función del número de relaciones que participan en la operación. En este caso serán unitarias o binarias.

Otra clasificación las categoriza en operaciones fundamentales o básicas y operaciones adicionales o derivadas ya que pueden expresarse en función de las anteriores.

Veremos las operaciones según esta última clasificación haciendo uso del ejemplo que se cita en el Anexo I.

OPERACIONES FUNDAMENTALES

OPERACIONES UNITARIAS: Trabajan con 1 sola relación. Son la SELECCIÓN y la PROYECCIÓN

SELECCIÓN: Devuelve una nueva relación con las tuplas que cumplen con el predicado de la selección

σ (R)

Predicado

El predicado puede realizar comparación de atributos con valores fijos o puede realizar comparación entre atributos. En este último caso siempre se compararán los valores contenidos en los atributos de la misma tupla o fila.

Se podrán utilizar en la comparación:

- Operadores aritméticos: =, ≠, >, ≥, <, ≤
- Operadores lógicos: Y (AND, ∧), O (OR, ∨), NEGADO (NOT, ¬)

Ejemplos:

1) *Listar los alumnos cuyo legajo sea menor a 37000.*

Operación: En este caso obtendremos la información de la tabla ALUMNOS, entonces:

σ (ALUMNOS)

legajo < 37000

Resultado:

legajo	apellidoYnombres	Dirección	Teléfono
31392	Juárez, María	9 de Julio 74	477-7777
34578	García, Felipe	Zeballos 23	466-6666

2) *Listar las carreras con planes en el año 1995 o en el año 2008.*

Operación: En este caso la información la obtendremos de la tabla CARRERAS_PLANES

σ (PLANES)

año_plan = 1995 o año_plan = 2008

Resultado:

cod_depto	año_plan	denominación_plan	resolucion
ISI	1995	Ingeniería en Sistemas Plan 1995	r222
ISI	2008	Ingeniería en Sistemas Plan 2008	r333
IQ	1995	Ingeniería Química Plan 1995	R555
IQ	2008	Ingeniería Química Plan 2008	R666

PROYECCIÓN: Devuelve una nueva relación con los atributos (columnas) indicados en la proyección. Elimina las tuplas repetidas

π (R)

atrib₁, atrib₂,..., atrib_n

Por ejemplo:

1) *Listar los años en que se hayan formulado planes de estudio de las distintas carreras*

Operación: En este caso la información la obtendremos de la tabla PLANES

π (PLANES)

año_plan

Resultado:

año_plan
1985
1995
2008
2023

2) *Listar apellidoy nombres y teléfono de los profesores*

Operación: En este caso la información la obtendremos de la tabla CARRERAS_PLANES

π (PROFESORES)

apellidoy nombres, telefono

Resultado:

apellidoy nombres	teléfono
Días, Carolina	438-0909
García, Felipe	466-6666
González, Luis	467-8899
Martínez, José	455-5555

OPERACIONES BINARIAS: Trabajan con 2 relaciones. Entre las operaciones fundamentales binarias encontraremos las que aplican la teoría de conjuntos: PRODUCTO CARTESIANO, UNION y DIFERENCIA.

PRODUCTO CARTESIANO: Aplicado a dos relaciones de grado N y M respectivamente, devuelve una nueva relación de grado N + M resultante de combinar todas las tuplas de la primer relación con las tuplas de la segunda relación (todas las columnas de la primera y la segunda). La cantidad de tuplas de la relación resultante será el producto de la cantidad de tuplas de ambas relaciones combinadas.

$$R_1 \quad X \quad R_2$$

Por ejemplo: *Realizar el producto cartesiano entre departamentos y planes de la carrera*

Operación:

$$\text{DEPARTAMENTOS} \quad X \quad \text{PLANES}$$

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

Resultado:

DEPARTAMENTOS.cod_depto	denominación	PLANES.cod_depto	año_plan	carrera	resolución
IQ	Ingeniería Química	ISI	1985	Ingeniería en Sistemas Plan 1985	r111
IQ	Ingeniería Química	ISI	1995	Ingeniería en Sistemas Plan 1995	r222
IQ	Ingeniería Química	ISI	2008	Ingeniería en Sistemas Plan 2008	r333
IQ	Ingeniería Química	ISI	2023	Ingeniería en Sistemas Plan 2023	r444
IQ	Ingeniería Química	IQ	1995	Ingeniería Química Plan 1995	R555
IQ	Ingeniería Química	IQ	2008	Ingeniería Química Plan 2008	R666
ISI	Ingeniería en Sistemas	ISI	1985	Ingeniería en Sistemas Plan 1985	r111
ISI	Ingeniería en Sistemas	ISI	1995	Ingeniería en Sistemas Plan 1995	r222
ISI	Ingeniería en Sistemas	ISI	2008	Ingeniería en Sistemas Plan 2008	r333
ISI	Ingeniería en Sistemas	ISI	2023	Ingeniería en Sistemas Plan 2023	r444
ISI	Ingeniería en Sistemas	IQ	1995	Ingeniería Química Plan 1995	R555
ISI	Ingeniería en Sistemas	IQ	2008	Ingeniería Química Plan 2008	R666
CB	Ciencias Básicas	ISI	1985	Ingeniería en Sistemas Plan 1985	r111
CB	Ciencias Básicas	ISI	1995	Ingeniería en Sistemas Plan 1995	r222
CB	Ciencias Básicas	ISI	2008	Ingeniería en Sistemas Plan 2008	r333
CB	Ciencias Básicas	ISI	2023	Ingeniería en Sistemas Plan 2023	R444
CB	Ciencias Básicas	IQ	1995	Ingeniería Química Plan 1995	R555
CB	Ciencias Básicas	IQ	2008	Ingeniería Química Plan 2008	R666

Como vemos el resultado no nos da información coherente. Ahora, si nos concentramos en aquellas tuplas donde DEPARTAMENTO.cod_depto = PLANES.cod_depto (marcadas en gris) podríamos obtener el nombre del departamento al que pertenece el plan de carrera y entonces tendría sentido la información.

La operación sería entonces:

$$\sigma_{\text{(DEPARTAMENTOS } \bowtie \text{ PLANES)}}$$

$$\text{DEPARTAMENTOS.cod_depto} = \text{PLANES.cod_depto}$$

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

Resultado:

DEPARTAMENTOS.cod_depto	denominación	PLANES.cod_depto	año_plan	carrera	resolución
IQ	Ingeniería Química	IQ	1995	Ingeniería Química Plan 1995	R555
IQ	Ingeniería Química	IQ	2008	Ingeniería Química Plan 2008	R666
ISI	Ingeniería en Sistemas	ISI	1985	Ingeniería en Sistemas Plan 1985	r111
ISI	Ingeniería en Sistemas	ISI	1995	Ingeniería en Sistemas Plan 1995	r222
ISI	Ingeniería en Sistemas	ISI	2008	Ingeniería en Sistemas Plan 2008	r333
ISI	Ingeniería en Sistemas	ISI	2023	Ingeniería en Sistemas Plan 2023	R444

Podríamos obtener una tabla sin los atributos cod_depto repetidos de la siguiente forma:

$$\pi \left(\sigma_{\text{DEPARTAMENTOS.cod_depto} = \text{PLANES.cod_depto}} (\text{DEPARTAMENTOS} \times \text{PLANES}) \right)$$

DEPARTAMENTOS.cod_depto = PLANES.cod_depto

DEPARTAMENTOS.cod_depto, denominación, año_plan, carrera, resolución

Resultado:

DEPARTAMENTOS.cod_depto	denominación	año_plan	carrera	resolución
IQ	Ingeniería Química	1995	Ingeniería Química Plan 1995	R555
IQ	Ingeniería Química	2008	Ingeniería Química Plan 2008	R666
ISI	Ingeniería en Sistemas	1985	Ingeniería en Sistemas Plan 1985	r111
ISI	Ingeniería en Sistemas	1995	Ingeniería en Sistemas Plan 1995	r222
ISI	Ingeniería en Sistemas	2008	Ingeniería en Sistemas Plan 2008	r333
ISI	Ingeniería en Sistemas	2023	Ingeniería en Sistemas Plan 2023	R444

Veremos más adelante como simplificar esta operación.

UNIÓN: Devuelve una nueva relación con las tuplas de la primer relación y las tuplas de la segunda relación. Se eliminan, como en todo conjunto, las tuplas repetidas.

$$R_1 \quad \mathbf{U} \quad R_2$$

Para aplicar la UNION las relaciones R_1 y R_2 deben ser **UNION COMPATIBLES**.

Diremos que dos relaciones R_1 y R_2 son UNION COMPATIBLES si tienen el mismo grado (misma cantidad de atributos) y el dominio de los valores del i -ésimo atributo de la relación R_1 es el mismo que el del i -ésimo atributo de la relación R_2 .

Por ejemplo: *Listar las personas de la institución: alumnos y profesores indicando apellido y nombre y teléfono.*

Si bien las relaciones ALUMNOS y PROFESORES tienen el mismo grado (4) y algunos de los atributos tienen el mismo dominio de valores, no sucede lo mismo con legajo de alumnos y nro_prof de profesores. Por lo tanto trabajaremos ambas relaciones para obtener relaciones unión compatibles y luego aplicaremos el operador UNION. Podemos realizar todo en la misma operación:

Operación:

$$\pi \text{ (ALUMNOS)} \quad \mathbf{U} \quad \pi \text{ (PROFESORES)}$$

apellidoynombres, teléfono apellidoynombres, teléfono

Resultado:

apellidoYnombres	teléfono
Juárez, María	477-7777
García, Felipe	466-6666
Pérez, Sofía	444-4444
Martínez, José	455-5555
Días, Carolina	438-0909
García, Felipe	466-6666
González, Luis	467-8899

Notar que **Martínez, José**, si bien está en ambas tablas (ALUMNOS Y PROFESORES) aparece una única vez en la relación resultante ya que se han eliminado las tuplas repetidas.

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

DIFERENCIA: Devuelve una nueva relación donde las tuplas están en la primera relación y no en la segunda. Las relaciones también deben ser unión compatibles (deben tener el mismo grado y sus atributos el mismo dominio de valores)

Por ejemplo: *Listar los ALUMNOS que no sean PROFESORES.*

Al igual que en el caso anterior tendremos que trabajar con relaciones UNION COMPATIBLES. Como el enunciado lo permite, trabajaremos solo con el atributo apellidoYnombres

Operación:

$$\pi(\text{ALUMNOS}) - \pi(\text{PROFESORES})$$

apellidoYnombres apellidoYnombres

Resultado:

apellidoYnombres
Juárez, María
García, Felipe
Pérez, Sofía

OPERACIONES ADICIONALES O DERIVADAS

INTERSECCIÓN: Devuelve una nueva relación con las tuplas que se encuentran en ambas relaciones. Se dice una operación adicional o derivada ya que puede resolverse en función de las operaciones básicas vistas anteriormente. También las relaciones involucradas deben ser unión compatibles.

$$R_1 \cap R_2 = R_1 - (R_1 - R_2)$$

Por ejemplo: *Listar las personas de la institución que sean ALUMNOS y PROFESORES.*

Al igual que en el caso anterior tendremos que trabajar con relaciones UNION COMPATIBLES. Como el enunciado lo permite, trabajaremos solo con el atributo apellidoYnombres

Operación:

$$\pi_{\text{apellidoYnombres}}(R_1) \cap \pi_{\text{apellidoYnombres}}(R_2)$$

Resultado:

apellidoYnombres
Martínez, José

NATURAL JOIN (REUNIÓN NATURAL O PRODUCTO NATURAL): Operación derivada de realizar el PRODUCTO CARTESIANO entre las relaciones involucradas, seguido de las operaciones SELECCIÓN y PROYECCIÓN. Estas últimas operaciones se realizan sobre los atributos del mismo dominio que se igualan para la selección y luego se quita el atributo repetido en la proyección.

Para evitar confusiones se solicita que los atributos tengan los mismos nombres en ambas relaciones, por lo que, si no los tienen, deberán renombrarse (Se explica en...)

$$R_1 \bowtie R_2$$

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

Ejemplo: *Listar los alumnos de la institución y los planes de carrera a las que están inscriptos.*

Operación:



Entre ALUMNOS y ALUMNOS_PLANES el atributo común que permite el PRODUCTO NATURAL es **legajo**. De la relación resultante de este PRODUCTO NATURAL, los atributos comunes que permiten realizar el PRODUCTO NATURAL son **cod_depto** y **año_plan**.

Resultado:

legajo	apellidoYnombres	dirección	teléfono	cod_depto	año_plan	fecha_insc	carrera	resolucion
31392	Juárez, María	9 de Julio 74	477-7777	IQ	1995	05/02/2006	Ingeniería Química Plan 1995	R555
31392	Juárez, María	9 de Julio 74	477-7777	ISI	1995	01/12/2004	Ingeniería en Sistemas Plan 1995	r222
31392	Juárez, María	9 de Julio 74	477-7777	ISI	2023	01/03/2023	Ingeniería en Sistemas Plan 2023	r444
34578	García, Felipe	Zeballos 23	466-6666	IQ	2008	01/11/2009	Ingeniería Química Plan 2008	R666
34578	García, Felipe	Zeballos 23	466-6666	ISI	2008	07/02/2011	Ingeniería en Sistemas Plan 2008	r333
37556	Pérez, Sofía	Entre Ríos 21	444-4444	ISI	2008	08/02/2011	Ingeniería en Sistemas Plan 2008	r333
38765	Martínez, José	Corrientes 22	455-5555	IQ	2008	03/12/2012	Ingeniería Química Plan 2008	R666
38765	Martínez, José	Corrientes 22	455-5555	ISI	2008	05/02/2013	Ingeniería en Sistemas Plan 2008	r333

JOIN (REUNIÓN THETA – THETA JOIN): Operación que permite realizar una combinación entre las relaciones que deriva de realizar el PRODUCTO CARTESIANO entre las mismas seguido de la operación SELECCIÓN cuya condición (θ) se debe respetar en la combinación.

$$R_1 \bowtie_{\theta} R_2$$

El ejemplo anterior: *Listar los alumnos de la institución y los planes de carrera a las que están inscriptos*, podría notarse:

Operación:

$$\begin{array}{ccc} \text{ALUMNOS} & \bowtie & \text{ALUMNOS_PLANES} & \bowtie & \text{PLANES} \\ \text{legajo} = \text{legajo} & & \text{cod_depto} = \text{cod_depto} \text{ Y} & & \\ & & \text{año_plan} = \text{año_plan} & & \end{array}$$

En este caso en que la condición se da por igual, el JOIN se denomina **EQUI-JOIN**.

No obstante, al igual que en la selección, las condiciones pueden ser cualquier expresión.

(Ver Anexo II: Ejemplo de combinación complejo.)

DIVISION: Supongamos una relación R_1 con un conjunto de atributos Z y una relación R_2 con un conjunto de atributos X . La operación división puede aplicarse entre R_1 y R_2 y se nota:

$$R_1 \div R_2$$

si $X \subseteq Z$ y dará como resultado una nueva relación con un conjunto de atributos Y donde $Y = Z - X$, es decir $Z = X \cup Y$, o lo que es lo mismo, Y serán los atributos de R_1 que no son atributos de R_2 .

Además, para que una tupla aparezca en la nueva relación los valores de la tupla deben estar en R_1 en combinación con todas las tuplas de R_2 .

La división es una operación derivada ya que puede expresarse como una secuencia de operaciones de PROYECCIÓN, PRODUCTO CARTESIANO y DIFERENCIA, como sigue:

$$T_1 \leftarrow \pi_Y (R_1)$$

$$T_2 \leftarrow T_1 \times R_2$$

$$T_3 \leftarrow T_2 - R_1$$

$$T_4 \leftarrow \pi_Y (T_3)$$

$$T \leftarrow T_1 - T_4$$

En forma práctica, utilizaremos la división cuando necesitemos responder a “**para todos**”

Ejemplo: *Listar todos los alumnos que se hayan inscripto a todas las carreras que se inscribió el alumno de legajo: 34578.*

Operación:

Primero obtendremos las carreras a las que se inscribió dicho alumno, que están dadas por cod_depto y año_plan de la tabla ALUMNOS_PLANES

$$R_2 \leftarrow \pi(\sigma_{\text{legajo} = 34578}(\text{ALUMNOS_PLANES}))$$

legajo = 34578

cod_depto, año_plan

R₂

cod_depto	año_plan
IQ	2008
ISI	2008

Para poder realizar la división deberemos obtener una tabla con las condiciones indicadas anteriormente por lo que:

$$R_1 \leftarrow \pi(\text{ALUMNOS_PLANES})$$

legajo, cod_depto, año_plan

legajo	cod_depto	año_plan
31392	IQ	1995
31392	ISI	1995
31392	ISI	2023
34578	IQ	2008
34578	ISI	2008
37556	ISI	2008
38765	ISI	2008
38765	IQ	2008

Al realizar la división: $R_1 \div R_2$, obtendremos el siguiente resultado

legajo
34578
38765

Siguiendo las operaciones fundamentales en los que se basa la división podríamos obtener:

$$T_1 \leftarrow \pi(R_1)$$

legajo

T₁

legajo
31392
34578
37556
38765

$$T_2 \leftarrow T_1 \times R_2$$

T₂

legajo	cod_depto	año_plan
31392	IQ	2008
34578	IQ	2008
37556	IQ	2008
38765	IQ	2008
31392	ISI	2008
31392	ISI	2023
34578	ISI	2008
37556	ISI	2008
38765	ISI	2008

$$T_3 \leftarrow T_2 - R_1$$

$$T_4 \leftarrow \pi(T_3)$$

Legajo

T₃

legajo	cod_depto	año_plan
31392	IQ	2008
37556	IQ	2008
31392	ISI	2008

T₄

legajo
31392
37556

$$T \leftarrow T_1 - T_4$$

T

legajo
34578
38765

Igual resultado que el obtenido anteriormente

RENOMBRAR:

Se han desarrollado hasta aquí operaciones anidadas a las que no hemos asignado un nombre y algunas operaciones a las que les hemos asignado un nombre para reflejar las operaciones intermedias, como por ejemplo en el caso de la DIVISIÓN.

Para renombrar bastará con indicar el nombre de la relación y los atributos que deseemos renombrar de la siguiente forma:

$$R_1 (\text{atributo}_{R_{11}}, \text{atributo}_{R_{12}}, \dots, \text{atributo}_{R_{1N}}) \leftarrow \pi(R) \quad \text{atributo1, atributo2, ..., atributoN}$$

INSERCIÓN – ELIMINACIÓN - ACTUALIZACIÓN

INSERCIÓN:

El agregado de tuplas a una relación se consigue mediante el operador **UNION**

$$R_1 \leftarrow R_1 \cup R_2$$

A partir de esta operación las tuplas de R_2 pasarán a formar parte de las de R_1 , respetando los conceptos de la teoría de conjunto, es decir, ambas relaciones deberán ser unión compatibles y se eliminarán las filas repetidas que hubiera.

Ejemplo:

Agregar a la tabla de departamentos el departamento IE: Ingeniería Eléctrica

Operación:

$$\text{DEPARTAMENTOS} \leftarrow \text{DEPARTAMENTOS} \cup \{('IE', 'Ingeniería Eléctrica')\}$$

Resultado:

DEPARTAMENTOS

cod_depto	denominación
IQ	Ingeniería Química
ISI	Ingeniería en Sistemas
CB	Ciencias Básicas
IE	Ingeniería Eléctrica

ELIMINACIÓN:

El borrado de tuplas de una relación se consigue mediante el operador **DIFERENCIA**

$$R_1 \leftarrow R_1 - R_2$$

A partir de esta operación las tuplas de R_2 dejarán de formar parte de las de R_1 , respetando los conceptos de la teoría de conjunto, es decir, ambas relaciones deberán ser unión compatibles y las del modelo relacional. En este último caso no se podrán eliminar filas que estén relacionadas con otras tablas a través de las claves foráneas.

Ejemplo:

Eliminar de la tabla de departamentos el departamento de código CB

Operación:

DEPARTAMENTOS $\leftarrow$ DEPARTAMENTOS - σ (DEPARTAMENTOS)

cod_depto = 'CB'

Resultado (partiendo de la tabla del ejemplo anterior):

DEPARTAMENTOS

cod_depto	denominación
IQ	Ingeniería Química
ISI	Ingeniería en Sistemas
IE	Ingeniería Eléctrica

No podríamos eliminar los departamentos de cod_depto: 'IQ' o 'ISI' ya que infringiríamos la regla de integridad referencial con la tabla PLANES

ACTUALIZACIÓN

$A \leftarrow \delta$ atributo = calculo (A)

ANEXO I – EJEMPLO MODELO RELACIONAL CON DATOS

DEPARTAMENTOS (Cod_depto, denominación)

CP

CF(DEPARTAMENTOS) NOT NULL

PLANES (Cod_depto, año_plan, denominación_plan, resolución)

CP

ALUMNOS (legajo, apellidoYnombres, dirección, teléfono)

CP

CF(ALUMNOS)

CF(CARRERAS_PLANES)

NOT NULL

NOT NULL

ALUMNOS_PLANES (legajo, cod_depto, año_plan, fecha_insc)

CP

PROFESORES (legajo, apellidoYnombres, dirección, teléfono)

CP

DEPARTAMENTOS

<u>cod_depto</u>	denominación
IQ	Ingeniería Química
ISI	Ingeniería en Sistemas
CB	Ciencias Básicas

PLANES

<u>cod_depto</u>	<u>año_plan</u>	carrera	resolucion
IQ	1995	Ingeniería Química Plan 1995	R555
IQ	2008	Ingeniería Química Plan 2008	R666
ISI	1985	Ingeniería en Sistemas Plan 1985	r111
ISI	1995	Ingeniería en Sistemas Plan 1995	r222
ISI	2008	Ingeniería en Sistemas Plan 2008	r333
ISI	2023	Ingeniería en Sistemas Plan 2023	r444

ALUMNOS

<u>legajo</u>	apellidoYnombres	dirección	teléfono
31392	Juárez, María	9 de Julio 74	477-7777
34578	García, Felipe	Zeballos 23	466-6666
37556	Pérez, Sofía	Entre Ríos 21	444-4444
38765	Martínez, José	Corrientes 22	455-5555

ALUMNOS_PLANES

legajo	cod_depto	año_plan	fecha_insc
31392	IQ	1995	05/02/2006
31392	ISI	1995	01/12/2004
31392	ISI	2023	01/03/2023
34578	IQ	2008	01/11/2009
34578	ISI	2008	07/02/2011
37556	ISI	2008	08/02/2011
38765	ISI	2008	03/12/2012
38765	IQ	2008	05/02/2013

PROFESORES

nro_prof	apellidoYnombres	dirección	teléfono
1	Días, Carolina	Entre Ríos 1999	438-0909
2	García, Felipe	Zeballos 23	466-6666
3	González, Luis	San Juan 2450	467-8899
4	Martínez, José	Corrientes 22	455-5555

ANEXO II – EJEMPLO DE COMBINACIÓN COMPLEJO

Supongamos el siguiente ejemplo: *Listar la última fecha de inscripción de los alumnos a cualquiera de los planes de carrera de la institución.*

Visto que no podemos realizar una comparación entre dos tuplas sucesivas de una tabla, realizaremos una combinación por mayor.

Para entender el ejemplo lo realizaremos paso a paso.

Primero utilizando PRODUCTO CARTESIANO y SELECCIÓN y luego convirtiendo esas operaciones utilizando operaciones derivadas.

Operaciones:

R1 ← ALUMNOS_PLANES

R2 ← ALUMNOS_PLANES

R3 ← σ (R1 X R2)
R1.legajo = R2.legajo
Y
R1.fecha_insc > R2.fecha_insc

En estas operaciones:

- Se han renombrado las tablas para no producir un resultado incorrecto y poder comparar los atributos que provienen de la misma tabla
- El producto cartesiano produce una nueva relación de grado 8 y 64 filas.
- De dicho producto cartesiano nos quedamos con aquellas filas que coincidan con el predicado de la selección. Es decir, solo nos interesarán las tuplas donde coincida el legajo y la fecha de inscripción de R1 sea > que la fecha de inscripción de R2 (Resaltados en gris en la tabla)

Resultado:

R1.legajo	R1.cod_depto	R1.año_plan	R1.fecha_insc	R2.legajo	R2.cod_depto	R2.año_plan	R2.fecha_insc
31392	IQ	1995	05/02/2006	31392	IQ	1995	05/02/2006
31392	IQ	1995	05/02/2006	31392	ISI	1995	01/12/2004
31392	IQ	1995	05/02/2006	31392	ISI	2023	01/03/2023
31392	IQ	1995	05/02/2006	34578	IQ	2008	01/11/2009
31392	IQ	1995	05/02/2006	34578	ISI	2008	07/02/2011

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

31392	IQ	1995	05/02/2006	37556	ISI	2008	08/02/2011
31392	IQ	1995	05/02/2006	38765	ISI	2008	03/12/2012
31392	IQ	1995	05/02/2006	38765	IQ	2008	05/02/2013
31392	ISI	1995	01/12/2004	31392	IQ	1995	05/02/2006
31392	ISI	1995	01/12/2004	31392	ISI	1995	01/12/2004
31392	ISI	1995	01/12/2004	31392	ISI	2023	01/03/2023
31392	ISI	1995	01/12/2004	34578	IQ	2008	01/11/2009
31392	ISI	1995	01/12/2004	34578	ISI	2008	07/02/2011
31392	ISI	1995	01/12/2004	37556	ISI	2008	08/02/2011
31392	ISI	1995	01/12/2004	38765	ISI	2008	03/12/2012
31392	ISI	1995	01/12/2004	38765	IQ	2008	05/02/2013
31392	ISI	2023	01/03/2023	31392	IQ	1995	05/02/2006
31392	ISI	2023	01/03/2023	31392	ISI	1995	01/12/2004
31392	ISI	2023	01/03/2023	31392	ISI	2023	01/03/2023
31392	ISI	2023	01/03/2023	34578	IQ	2008	01/11/2009
31392	ISI	2023	01/03/2023	34578	ISI	2008	07/02/2011
31392	ISI	2023	01/03/2023	37556	ISI	2008	08/02/2011
31392	ISI	2023	01/03/2023	38765	ISI	2008	03/12/2012
31392	ISI	2023	01/03/2023	38765	IQ	2008	05/02/2013
34578	IQ	2008	01/11/2009	31392	IQ	1995	05/02/2006
34578	IQ	2008	01/11/2009	31392	ISI	1995	01/12/2004
34578	IQ	2008	01/11/2009	31392	ISI	2023	01/03/2023
34578	IQ	2008	01/11/2009	34578	IQ	2008	01/11/2009
34578	IQ	2008	01/11/2009	34578	ISI	2008	07/02/2011
34578	IQ	2008	01/11/2009	37556	ISI	2008	08/02/2011
34578	IQ	2008	01/11/2009	38765	ISI	2008	03/12/2012
34578	IQ	2008	01/11/2009	38765	IQ	2008	05/02/2013

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

34578	ISI	2008	07/02/2011	31392	IQ	1995	05/02/2006
34578	ISI	2008	07/02/2011	31392	ISI	1995	01/12/2004
34578	ISI	2008	07/02/2011	31392	ISI	2023	01/03/2023
34578	ISI	2008	07/02/2011	34578	IQ	2008	01/11/2009
34578	ISI	2008	07/02/2011	34578	ISI	2008	07/02/2011
34578	ISI	2008	07/02/2011	37556	ISI	2008	08/02/2011
34578	ISI	2008	07/02/2011	38765	ISI	2008	03/12/2012
34578	ISI	2008	07/02/2011	38765	IQ	2008	05/02/2013
37556	ISI	2008	08/02/2011	31392	IQ	1995	05/02/2006
37556	ISI	2008	08/02/2011	31392	ISI	1995	01/12/2004
37556	ISI	2008	08/02/2011	31392	ISI	2023	01/03/2023
37556	ISI	2008	08/02/2011	34578	IQ	2008	01/11/2009
37556	ISI	2008	08/02/2011	34578	ISI	2008	07/02/2011
37556	ISI	2008	08/02/2011	37556	ISI	2008	08/02/2011
37556	ISI	2008	08/02/2011	38765	ISI	2008	03/12/2012
37556	ISI	2008	08/02/2011	38765	IQ	2008	05/02/2013
38765	ISI	2008	03/12/2012	31392	IQ	1995	05/02/2006
38765	ISI	2008	03/12/2012	31392	ISI	1995	01/12/2004
38765	ISI	2008	03/12/2012	31392	ISI	2023	01/03/2023
38765	ISI	2008	03/12/2012	34578	IQ	2008	01/11/2009
38765	ISI	2008	03/12/2012	34578	ISI	2008	07/02/2011
38765	ISI	2008	03/12/2012	37556	ISI	2008	08/02/2011
38765	ISI	2008	03/12/2012	38765	ISI	2008	03/12/2012
38765	ISI	2008	03/12/2012	38765	IQ	2008	05/02/2013
38765	IQ	2008	05/02/2013	31392	IQ	1995	05/02/2006
38765	IQ	2008	05/02/2013	31392	ISI	1995	01/12/2004
38765	IQ	2008	05/02/2013	34578	IQ	2008	01/11/2009
38765	IQ	2008	05/02/2013	31392	ISI	2023	01/03/2023
38765	IQ	2008	05/02/2013	34578	ISI	2008	07/02/2011

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

38765	IQ	2008	05/02/2013	37556	ISI	2008	08/02/2011
38765	IQ	2008	05/02/2013	38765	ISI	2008	03/12/2012
38765	IQ	2008	05/02/2013	38765	IQ	2008	05/02/2013

R3 queda entonces:

R1.legajo	R1.cod_depto	R1.año_plan	R1.fecha_insc	R2.legajo	R2.cod_depto	R2.año_plan	R2.fecha_insc
31392	IQ	1995	05/02/2006	31392	ISI	1995	01/12/2004
31392	ISI	2023	01/03/2023	31392	IQ	1995	05/02/2006
31392	ISI	2023	01/03/2023	31392	ISI	1995	01/12/2004
34578	ISI	2008	07/02/2011	34578	IQ	2008	01/11/2009
38765	IQ	2008	05/02/2013	38765	ISI	2008	03/12/2012

A partir de R3 nos quedamos con las columnas que refieren a las inscripciones con menor fecha, es decir las columnas que eran de R2

$R4 \leftarrow \pi(R3)$

R2.legajo, R2.cod_depto, R2.año_plan, R2.fecha_insc

R2.legajo	R2.cod_depto	R2.año_plan	R2.fecha_insc
31392	ISI	1995	01/12/2004
31392	IQ	1995	05/02/2006
34578	IQ	2008	01/11/2009
38765	ISI	2008	03/12/2012

BASES DE DATOS - Unidad 2: Diseño de Bases de Datos - Modelo Relacional

Si realizamos la siguiente operación:

ALUMNOS_PLANES – R4

El resultado nos dará las inscripciones con mayor fecha de cada uno de los alumnos.

legajo	cod_depto	año_plan	fecha_insc
31392	ISI	2023	01/03/2023
34578	ISI	2008	07/02/2011
37556	ISI	2008	08/02/2011
38765	IQ	2008	05/02/2013